

ASMIT SAHU

23052231

✓ CNN (CONVOLUTION NURAL NETWORK)

important imports

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import classification_report
```

train test samples

```
(x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()
```

```
print("Training samples:", x_train.shape)
print("Test samples:", x_test.shape)
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ————— 4s 0us/step
Training samples: (50000, 32, 32, 3)
Test samples: (10000, 32, 32, 3)
```

```
x_train = x_train / 255.0
x_test = x_test / 255.0
```

giving class names for attributes

```
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']
```

```
plt.figure(figsize=(10,5))
for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(x_train[i])
    plt.title(class_names[y_train[i][0]])
    plt.axis('off')
plt.show()
```



Model Summary -

1. 3 convolution layers of 32,64,64 filters and kernel size of 3,3
2. 1 Flatten layer
3. 1 Fully connected layers of 64 neurons and 1 output layer to classify 10 items

```
model = models.Sequential()

model.add(layers.Conv2D(32,(3,3),activation='relu',input_shape=(32,32,3)))
model.add(layers.MaxPooling2D((2,2)))

model.add(layers.Conv2D(64,(3,3),activation='relu'))
model.add(layers.MaxPooling2D((2,2)))

model.add(layers.Conv2D(64,(3,3),activation='relu'))

model.add(layers.Flatten())

model.add(layers.Dense(64,activation='relu'))
model.add(layers.Dense(10,activation='softmax'))

model.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input\_shape`
 super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_1 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_2 (Conv2D)	(None, 4, 4, 64)	36,928
flatten (Flatten)	(None, 1024)	0
dense (Dense)	(None, 64)	65,600
dense_1 (Dense)	(None, 10)	650

Total params: 122,570 (478.79 KB)

Trainable params: 122,570 (478.79 KB)

Non-trainable params: 0 (0.00 B)

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

```
history = model.fit(
    x_train,
    y_train,
    epochs=15,
    validation_split=0.2,
    batch_size=64
)
```

```
Epoch 1/15
625/625 ————— 8s 7ms/step - accuracy: 0.3013 - loss: 1.8827 - val_accuracy: 0.5100 - val_loss: 1.3609
Epoch 2/15
625/625 ————— 3s 5ms/step - accuracy: 0.5272 - loss: 1.3238 - val_accuracy: 0.5693 - val_loss: 1.2113
Epoch 3/15
625/625 ————— 3s 5ms/step - accuracy: 0.5941 - loss: 1.1493 - val_accuracy: 0.6228 - val_loss: 1.0737
Epoch 4/15
625/625 ————— 3s 4ms/step - accuracy: 0.6297 - loss: 1.0518 - val_accuracy: 0.6394 - val_loss: 1.0336
Epoch 5/15
625/625 ————— 3s 4ms/step - accuracy: 0.6698 - loss: 0.9502 - val_accuracy: 0.6677 - val_loss: 0.9547
Epoch 6/15
625/625 ————— 3s 5ms/step - accuracy: 0.6890 - loss: 0.8855 - val_accuracy: 0.6610 - val_loss: 0.9711
Epoch 7/15
625/625 ————— 3s 5ms/step - accuracy: 0.7105 - loss: 0.8346 - val_accuracy: 0.6795 - val_loss: 0.9323
Epoch 8/15
625/625 ————— 3s 4ms/step - accuracy: 0.7275 - loss: 0.7825 - val_accuracy: 0.6825 - val_loss: 0.9399
Epoch 9/15
```

```

625/625 ————— 3s 4ms/step - accuracy: 0.7406 - loss: 0.7485 - val_accuracy: 0.6885 - val_loss: 0.9064
Epoch 10/15
625/625 ————— 3s 4ms/step - accuracy: 0.7568 - loss: 0.6955 - val_accuracy: 0.6988 - val_loss: 0.8769
Epoch 11/15
625/625 ————— 3s 5ms/step - accuracy: 0.7673 - loss: 0.6686 - val_accuracy: 0.6982 - val_loss: 0.8844
Epoch 12/15
625/625 ————— 3s 4ms/step - accuracy: 0.7837 - loss: 0.6306 - val_accuracy: 0.7028 - val_loss: 0.8677
Epoch 13/15
625/625 ————— 3s 4ms/step - accuracy: 0.7930 - loss: 0.5971 - val_accuracy: 0.7060 - val_loss: 0.8851
Epoch 14/15
625/625 ————— 3s 4ms/step - accuracy: 0.7991 - loss: 0.5748 - val_accuracy: 0.7072 - val_loss: 0.8805
Epoch 15/15
625/625 ————— 3s 4ms/step - accuracy: 0.8127 - loss: 0.5461 - val_accuracy: 0.7124 - val_loss: 0.8911

```

Accuracy metrics

```
test_loss, test_acc = model.evaluate(x_test,y_test)
```

```
print("Test Accuracy:",test_acc)
```

```

313/313 ————— 2s 2ms/step - accuracy: 0.7104 - loss: 0.9152
Test Accuracy: 0.7092999815940857

```

```

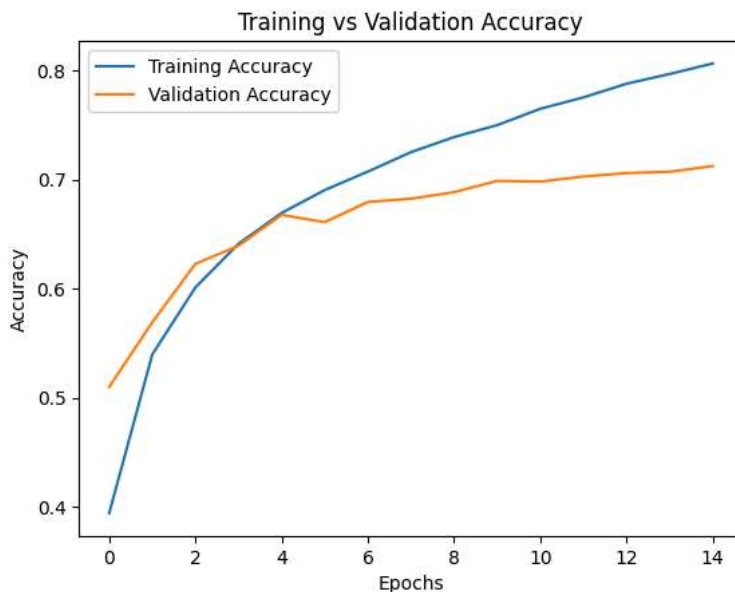
plt.figure()

plt.plot(history.history['accuracy'],label='Training Accuracy')
plt.plot(history.history['val_accuracy'],label='Validation Accuracy')

plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.title("Training vs Validation Accuracy")

plt.show()

```



```

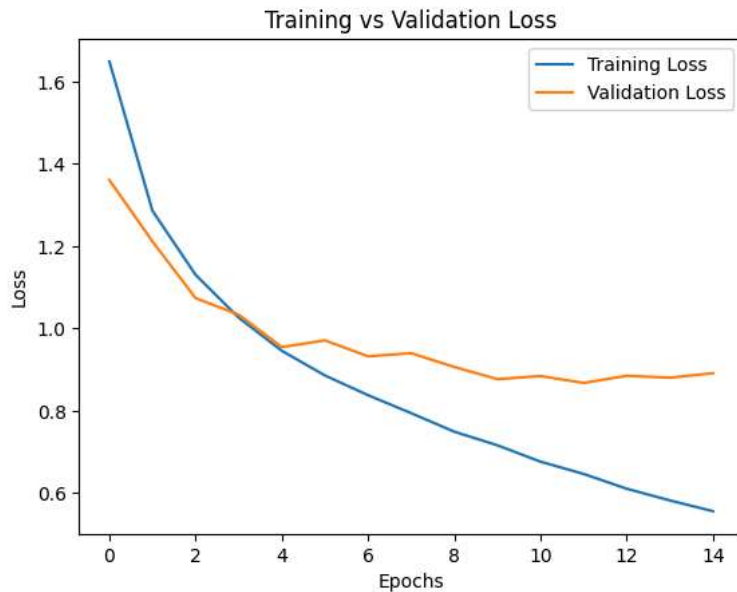
plt.figure()

plt.plot(history.history['loss'],label='Training Loss')
plt.plot(history.history['val_loss'],label='Validation Loss')

plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.title("Training vs Validation Loss")

plt.show()

```



```

predictions = model.predict(x_test)

predicted_labels = np.argmax(predictions,axis=1)
true_labels = y_test.flatten()

print(classification_report(true_labels,predicted_labels,target_names=class_names))

```

```

313/313 ————— 1s 2ms/step
          precision    recall  f1-score   support

   airplane      0.67      0.79      0.73      1000
  automobile      0.88      0.77      0.82      1000
         bird      0.66      0.55      0.60      1000
          cat      0.53      0.56      0.54      1000
         deer      0.68      0.65      0.66      1000
          dog      0.71      0.55      0.62      1000
         frog      0.71      0.83      0.76      1000
         horse      0.70      0.80      0.74      1000
          ship      0.78      0.82      0.80      1000
         truck      0.80      0.78      0.79      1000

 accuracy              0.71      10000
  macro avg      0.71      0.71      0.71      10000
 weighted avg      0.71      0.71      0.71      10000

```

```

plt.figure(figsize=(12,6))

for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(x_test[i])
    plt.title("Pred: "+class_names[predicted_labels[i]])
    plt.axis('off')

plt.show()

```

Pred: cat



Pred: ship



Pred: ship



Pred: airplane



Pred: frog



Pred: frog



Pred: truck



Pred: frog



Pred: cat



Pred: automobile

